



Video Editing Evaluation Using Auto-Supervised Learning

NJEIM Halim, VENTURINI Riccardo

May 4, 2026

Abstract

In the world of cinematography, video editing with the goal of attaining a satisfactory final product, especially in the film domain, is an exceptionally difficult and complex process, involving many secondary steps and tasks, each with its own field of study and expertise requirements. One of these tasks involves the joining of several scenes, each from a different source, so that their culmination forms a captivating, entertaining and instructive narrative that can be viewed as a story or at least a coherent sequence. In our research, we focused mainly on trying to incorporate the machine in the hopes of creating a model that can identify the quality of a cut, that is, what forms and constitutes each individual scene in a movie or a media sequence.

1 Introduction

Our research involved, in its majority and as a first step, understanding the main problem we were provided with. Because what does it mean to identify a "good" cut? First of all, what is even a cut? To answer this question we needed to dive into the world of cinematography rather than simply stay in the shallow waters of Visual Computing. Because cinematography is pervasive in all cultures and across all continents, it is only natural to try and automate it, or at least incorporate an element of Computer Science into it. But because Cinematography relies heavily on a director's final judgment [19] and on an alignment between a pre-existing script and the scenes taken, the task of completely automating the process (whether through editing, storytelling, shooting, directing, etc...) is still largely difficult. This study is based solely on visual analysis, without taking audio into account.

1.1 Motivation

Everybody likes film. And although it might not be dedicated devotion from everyone, many people enjoy the prospect of sitting down and enjoying a motion picture in its different forms and aspects. And everyone in turn has a tendency to develop an opinion based on a movie he or she watched, therefore, working as a part of the whole in the aim towards developing a tool with which creating better final products would be possible, was an interesting idea that pulled us to this project.

Our goal was to test a model's capabilities in establishing the building blocks of a cut, later deeming it as good or bad. At first, understanding what was required was not an easy task in itself, because the cut can be a simple migration of the camera, not necessarily a split between different scenes, for our main target was cuts used in single scenes rather than those used to separate between them. (A

scene takes place in one setting, ideally with the same set of characters). And because video and cinematography were not just field-specific tools, instead spanning over many domains (journalism, documentaries, entertainment, short-style content, etc...), our main focus was on cuts used specifically in movie scenes. More specifically, mainstream, box-office movies that were once presented in a theatre.

And so, to return to Computer Science terminology in order to dissect the problem, our work was related to Binary Classification [14]; basically a machine learning model with binary output. Of course, the ultimate goal of our research did not revolve around the idea of creating a finalizing model that would definitely identify any cut and correctly classify it, but to see whether this task was possible in the first place, and whether a model was capable of learning what took directors and filmmakers years of experience and a very distinct, talented gaze, to do. And because our work also included omission of audio and context from the data, it was the more curious.

1.2 Related works

Many studies explored the usage of machine learning in the process of making film, mainly in editing, where this study goes in depth [34]. This work involved the construction of an artificial scene using text from the scripts of the film, by selecting a set of keywords to build each scene [15]. More ambitious projects looked at creating a full video editing syntax through clip analysis and processing, in the hopes of building a final interface for implementing Artificial Intelligence technologies to edit videos [31], [32].

But none dealt directly with the problem we had at hand. Furthermore they [32] specified that most film editing techniques employed in the technology domain tackle the matter by the position of the actors in a scene, rather than the taking into consideration the wide range of stylistic decisions a director makes to come up with the final artistic product (shot transitions, scenes, angles, motion, etc...), which meant that the plethora of studies and research projects dealt directly with the content of the video data, from capturing emotions [28], identifying user engagement with the video content through analysis of their expression via recorded data [23], to detecting the subject speaker in video material [9].

Some works tackled the an approach based on the contextual flow of frames, images here [6]. Others aimed at the extraction of what were deemed as main events in a long video [7]. We found some studies that revolved around analyzing shot similarities by creating graphs that represent the shots and their boundaries as motion and color [22] and others that related the objects across multiple scenes as nodes with relations between them as the edges, ultimately yielding accurate real-world results [24].

Regardless of all the studies out there, our work still aimed at looking at the cut specifically, making the objects of the scenes, along with the angles, stylistic approaches and any cinematic input, for lack of a better word, irrelevant to our focused goal. Instead, we were to let the model decide on its own the various key features it will rely on to make the inferences and ultimately judge whether a cut was useable or not.

2 Data

To construct our dataset, we sourced video scenes from university archives, YouTube, and primarily from **Moviecuts**, an open-source GitHub project and research repository [21]. Globally, we had around 60 thousand video scenes.

2.1 Identifying the cut

In our specific case we are only looking for cinematic cuts made within the same scene, therefore we exclude cases of scene transitions.

As is discussed in [21], there are many different types of cuts to be considered in a movie. Briefly put, a Cut is a set of two adjacent shots as well as the transition between them. A great element of the cut was also the Audio that accompanied the frames which composed the Video.



Figure 1: Different cut types, source: [21]

But for our disadvantage, the consideration of audio in our tests would add multiple layers of both time and complexity in our research, which was impossible as time was somewhat limited. Because the cut was also a key component of the editing process, it was used more often as a tool to preserve the audio and visual continuity of the series of action displayed on the screen [25]. This type of detailed analysis required additional parts of the research; monitoring both the visual and the audio streams (as is shown in Figure 1), relating each speaker to the speech being played (in scenes where there is conversation), and finally adjusting the cut with the use of both streams.

Because not all scenes are conversation scenes, for there are voice over, action scenes, scenes that depend heavily on music (as in horror or drama) and a myriad of other types, taking audio into consideration was a task that required deeper research. This is why we preferred to exclude the audio from our research, plus it would prove more interesting to see the model achieve any kind of result with only the visual data.

2.2 Extracting the cuts

Before diving into our data labeling and research, we needed to extract each cut separately in order to then create the bad cuts first, then feed them to the model. This alone was a great task for several reasons.

1) Firstly, that our dataset was comprised of scenes which included many cuts each one of them, therefore necessitating a segregation. 2) Secondly, that the technologies used for cut detection and segregation were a whole field of research on their own. 3) Thirdly, because while old motion pictures of the twentieth century included merely cut scenes glued together to form the final product, newer movies and styles depended on a wide range of shot transition, especially nowadays with the major increase in budgets and technology (under the influence of what is described as Technological Determinism Theory) thus creating a large number of different transitional models which can be considered as a cut [8], [2].

For this purpose, we explored two main tools to help us with the extraction of our cuts from the scenes we had:

The first being TransNet [26], a deep network architecture used to detect shot transitions, specifically focusing on faster cuts which other technologies could not handle. The basic working of this model was that from a video it presented a `.txt` file with the list of frame numbers bordering a specific scene (i.e **0 104** would be scene 1 and **105 290** would be scene 2). The work would then

include using **OpenCv** [4] to first separate the scene into frames and then rejoining those frames - according to our own "bad cut creation" steps - to create a fresh video file which includes the shot.

In its paper, TransNet V2 (which was an improved version of the original [17]) was compared with other technologies involved in the same research (DeepSBD [10] and DSM [27]).

The second tool - which we ultimately settled for - was [PySceneDetect](#), an open-source shot transition detection tool, that is based on OpenCV and FFMPEG [20], [29]. The program detects breaks in between any content, with a capacity for detecting fade-to-black transitions.

It mainly uses differences in HSL colour space combined with filtering to detect shot changes.

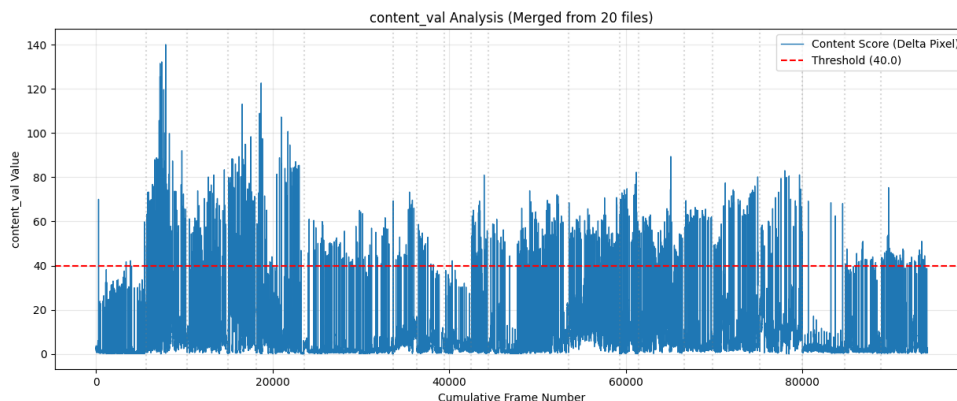


Figure 2: Pixel changes between frames of a scene and the threshold for cut detect

The reasons to why we settled for this tool were **1)** that it automatically segregated our cuts into **.mp4** file, facilitating the process of immediately creating the cuts and feeding them into our data loader, helping us evade the need to depend on OpenCv as another layer of complexity and margin of error when separating the video into frames and then rejoining them. And **2)** because the TransNet network was trained using datasets brought in from documentaries and homemade clips, rather than movie shot transitions, which we thought could bias our results towards the less budget-oriented films of the data by accurately segregating their cuts in favour of expensive, action-packed, box office projects.

2.3 Labeling the data

Labeling the data was our first theoretical challenge. Because none of us was by any means an expert in film and cinematography, deeming a cut to be good or bad was a mighty challenge. But for the sake of experimentation and science, we devised a simple, yet effective, approach:

To consider every officially-released movie scene to include only good cuts.

This, of course, presented us with another problem; risking that the model learns only by heart every individual cut presented in the scenes, therefore falling into overfitting. Nonetheless, it was our best opportunity at tackling the good part of our data, as getting into the minute details of identifying a cut requires research to which we couldn't find anything similar.

The videos were then sampled at 16 frames around the cut point (so 32 frames per sample) and fed to the model as a single stream.

2.4 Creating "Bad Cuts"

Due to our lack of expertise in cinematography and filmmaking, the same problem we had with the good cuts was present in creating the bad ones. Who can deem a cut to be bad, except a well-experienced director, even they err sometimes. And the human senses, as opposed to the machine, benefit from the jointure of auditory inputs in addition to the visual.

Initially, we discussed the possibility of scouring the internet, YouTube to be more specific, in search for a source of low quality films. We then stumbled upon a channel that makes low-budget,

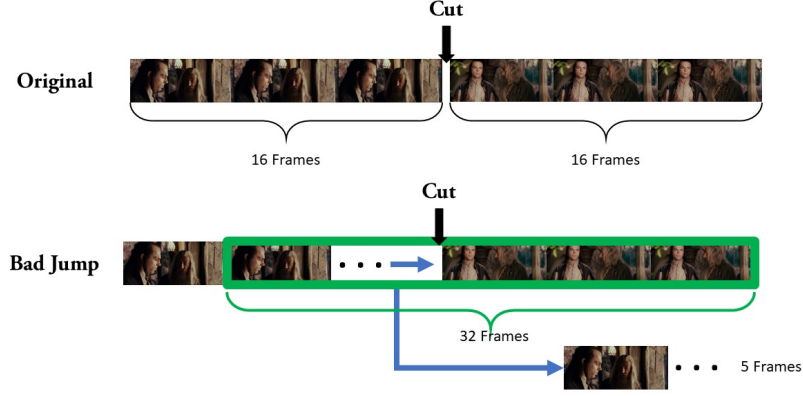


Figure 3: Creating bad jumps through pruning 5 frames before the original cut, thereby shifting the cut

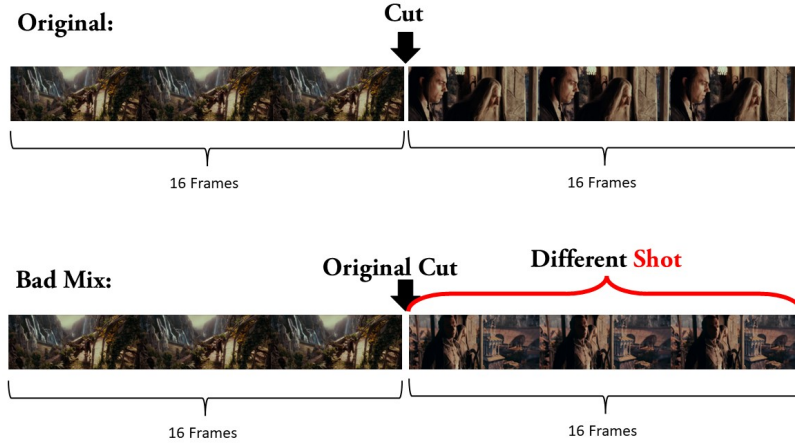


Figure 4: Creating bad mix through combining a different shot after the original cut

homemade content that are supposed to be movie scenes. And although any observer with common sense would deem their scenes to be less than professional, we were at risk of overfitting as well, leading the model to memorize that any cut that involved a home-like, small setting would be bad.

Eventually, we devised two main methods to artificially create the cuts from our already available dataset.

The first method involved shifting where a cut would originally be in a scene by 5 frames to the left (of the cut). We called these **Bad Jump Cuts**.

The second method was to randomly mix the shots before and after the cut. Specifically to substitute the original after shot with another random shot - from the same scene to preserve information (same actors, background, setting, etc...). We called these **Bad Mix Cuts**.

3 Building

For our project we chose to use the **ResNet 18** layers [11] model with **R2+1D** weights pre-trained on **Kinetics-400**[12]. The choice of the model was influenced by the positive results of MovieCuts project previously mentioned, because it deals with solving a problem similar to ours; creating a model that can classify cuts.

R2D+1D compared to R3D convolution has the advantage of lowering the error rate and being lighter, because in this case we separate the 2 spatial dimensions from the temporal dimension.[30]

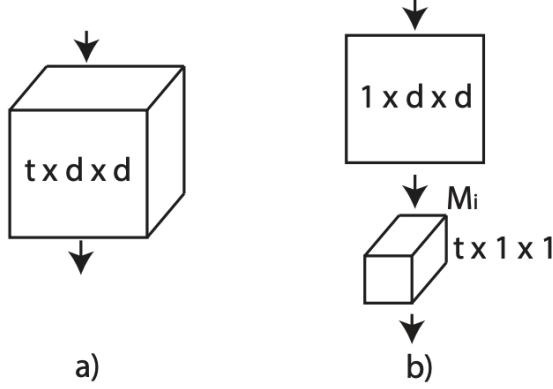


Figure 5: **(2+1)D vs 3D convolution.** The illustration is given for the simplified setting where the input consists of a spatiotemporal volume with a single feature channel. **(a)** Full 3D convolution is carried out using a filter of size $t \times d \times d$ where t denotes the temporal extent and d is the spatial width and height. **(b)** A (2+1)D convolutional block splits the computation into a spatial 2D convolution followed by a temporal 1D convolution. We choose the numbers of 2D filters (M_i) so that the number of parameters in our (2+1)D block matches that of the full 3D convolutional block.[30]

The model’s head is modified to output $N = 2$ classes, corresponding to ”Good” and ”Bad” cuts. The input to the network consists of **16 frames** sampled around the cut point.

We train the model using the **Adam optimizer**[13] with a learning rate of 10^{-4} . We utilize a **weighted CrossEntropyLoss** to address the lower specificity observed in baseline runs. The model is trained for up to 10 epochs with an effective **batch size of 4** on a single GPU. We implement an ”Early Saving” strategy, preserving the weights that achieve the minimum **Validation Loss** to ensure the best generalization performance. We created a dataset with **375754 cuts** of total from the videos obtained, that we called **First Dataset** (see chapter 2). To carry out some experiments we created a **Second Dataset**, which we will talk about in the chapter 4.

4 Experiments

The first training carried out gave very promising results, in fact we arrived to have a 83,52% of accuracy in validation phase:

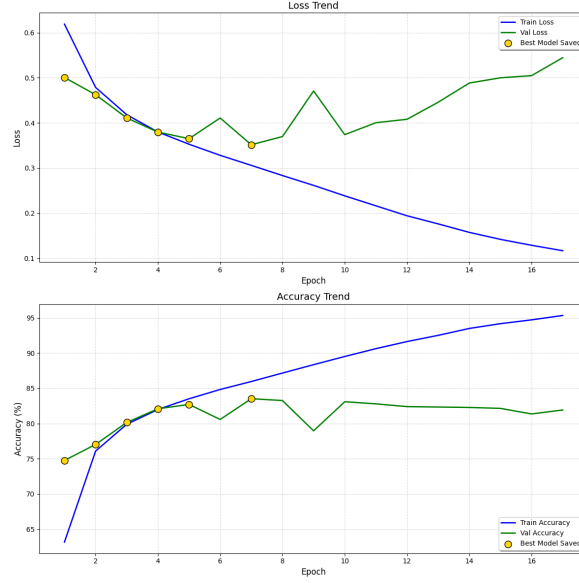


Figure 6: Training with building configuration on First Dataset.

Following the testing phase, analyzed in chapter 4.1.1, we realized that the real accuracy on new videos is much lower. The hypothesis is that there is a **Distribution Shifts** problem, so when the data distribution of an algorithm's training dataset differs from the distribution it encounters when deployed.[1] We think the model learned the compression flaws of the training dataset rather than the cuts themselves.

So a new data was downloaded, choosing random scenes, from the YouTube channel called [Movieclips](#). Creating the Second Dataset we got 14992 cuts (7496 good and 7496 bad), this time the bad cuts between bad jumps and bad mixes were split about 50/50.

We started a new training on the Second Dataset with same configuration:

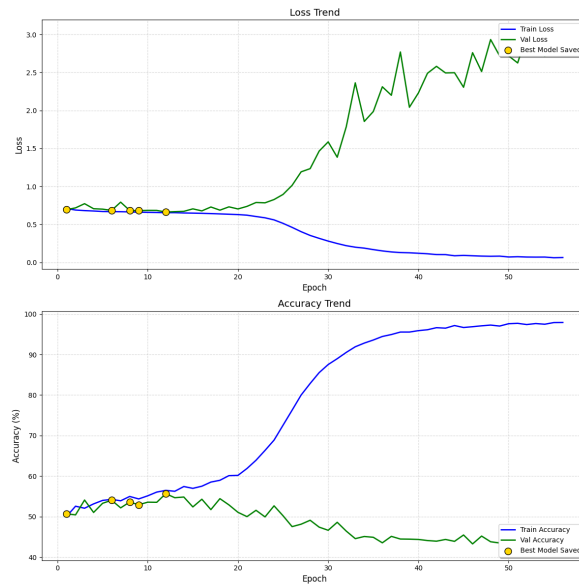


Figure 7: Training on the Second Dataset with the building configuration

As we can see after epoch 12 we went into overfitting, we can see it because in training the loss started to decrease a lot and in validation we have the opposite effect, so the model learned the videos

by heart without being able to generalize the rules. Furthermore in this case the validation accuracy was only 55% when we extracted the best model, so very low.

To prevent the model from memorizing pixel positions, we tried the **Data Augmentation** technique. This involves applying transformations to the videos while maintaining their optimal properties, and then applying **random resizing, spatial cropping, and color perturbation** operations, which force the neural network to focus on the semantic and dynamic characteristics of the crop rather than on static and repetitive visual patterns.[16]

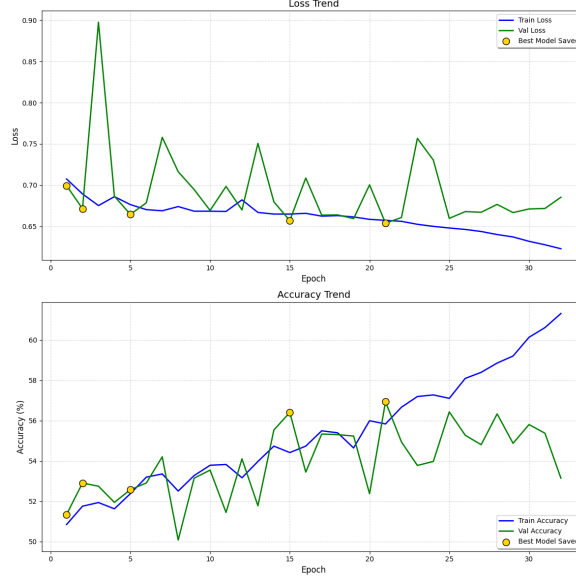


Figure 8: Training on the Second Dataset with Data Augmentation

In this case we obtain a slight overfitting, however the model failed to learn, reaching the maximum of 56% in validation. The problem in our case is that some manipulations on good cuts would turn them into bad cuts, but this would tell the model that it's still a good cut. So the result was as expected. *This was the only training with Data Augmentation.*

To counteract overfitting we tried to change the parameters, so this time using the AdamW[18] optimizer with a learning rate of 10^{-4} and a **L2 Regularization** (Weight Decay) of 0.01 to avoid excessive weight growth and ensure that the model does not become overly sensitive to noise within the training data. We used a ReduceLROnPlateau type **Learning Rate Scheduler** with 4 epochs of tolerance, to avoid that the model gets stuck in a stagnation phase or starts oscillating violently around a local minimum of the loss function without ever being able to converge. Additionally, we've added a **new class**, called "No_cuts", which are 32-frame video fragments without any cuts. This is called **negative sampling**[33] and was done to teach the model how visual continuity works. Finally we performed **Cost-Sensitive Learning** by weighting the classes. Since the model tends to predict that a video is a bad cut, we set the bad_cuts class to 2. The results seem promising but they hide a bias:

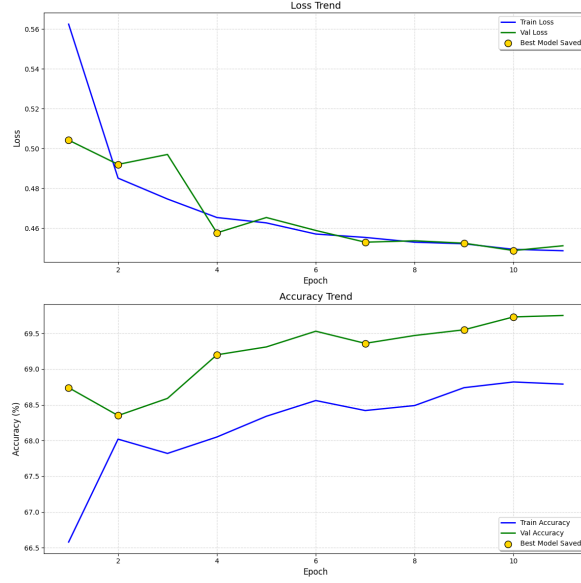


Figure 9: Training on new dataset with new No_cuts class

The problem is the way the three-class validation phase is performed. We noticed this during testing, the model learned the difference between a video with a cut and one without, but we're still far from a good result for the difference between positive and negative cuts. In fact, in this case, during testing, more than 90% of the time it said a model was negative. This is because during the training phase validation, the dataset was split incorrectly. Therefore, by mixing all three classes, the model only needed to find the videos without cuts and declare them to be bad to achieve good accuracy.

To try to force the model to find more robust patterns, we tried training with the **Dropout** technique, meaning that neurons were turned off for each training session [3]. In our case, for each training session, the neurons had a 50% chance of being turned off.

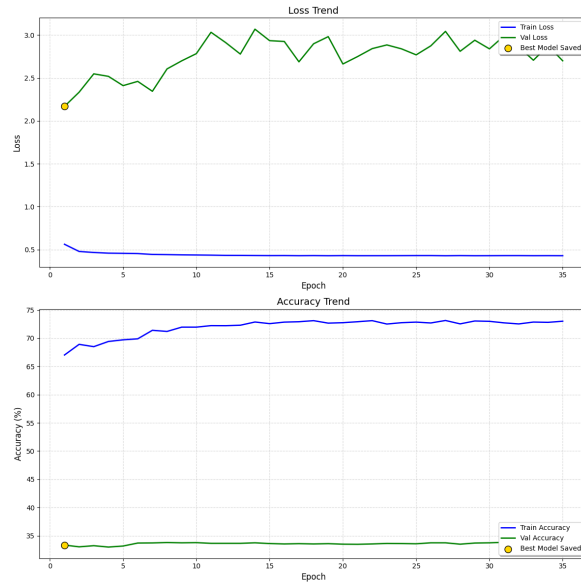


Figure 10: Training on the Second Dataset with Dropout

The difference between the training phase and the validation phase is evident, for 35 epochs the model had increasingly worse results in validation. We can see that the loss value in validation was

greater than 2, this means that the model is particularly doubtful.

So for the **final training** we merged the two datasets, we removed the dropout and we use same configuration of Fig 9 (Given the imbalance of the No Cuts class we have readjusted the weights accordingly).

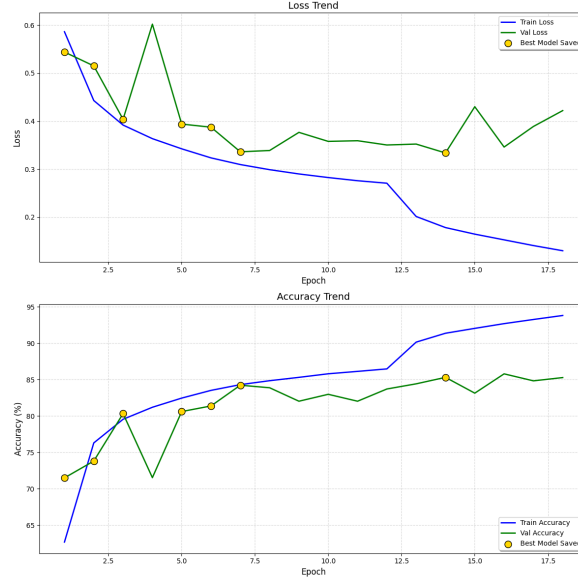


Figure 11: Training on all Dataset

In this case we obtained 85.31% as best Validation Accuracy and we had a good training values. In chapter 4.1.2 we will analyze the real results on new videos dataset.

4.1 Results

4.1.1 First training

The results refer to Fig 6, it was the most promising training among all those carried out. The model ran for 18 epochs and the epoch with the best values was **epoch 7** with **83.52 %** for **Validation Accuracy** and **85.96 %** for **Training Accuracy**. Therefore, this will be the model whose test results we will analyze.

What we did for testing was the following.

Create a batch of only good cuts, a batch of only bad cuts (50 % Bad jumps, 50 % Bad Mix), which yielded the following results:

- The model correctly identified **71 %** of the **Good cuts** as "good".
- The model correctly identified **33 %** of the **Bad cuts** as "bad".
- The model has an accuracy of **26 %** on **Bad Jumps** and the **40 %** on **Bad Mix**.

Clearly the model was better at identifying the bad mixed cuts instead of the ones where the jump was done prematurely by 5 frames.

We then did the same process of testing but with an **animated film**, to see whether the model had any biases towards this type. Again we received almost the same results from this set, with **70 %** of **Good cuts** correctly identified, and **38 %** of **Bad cuts** correctly identified.

Next, we repeated the process but using a single scene of film. Again, our results were similar: **70 %** of **Good cuts** correctly identified, and **31 %** of **Bad cuts** correctly identified.

We used the entire Second Dataset to calculate the accuracy of our model, comprised of **7495 cuts** per class (good and bad), which the model had never seen. Following the same division as above for the bad cuts (50 - 50), we ran the tests, yielding eventually the above matrix of confusion Table 1.

		Predicted Class		Total Real
		Positive	Negative	
Actual Class	Positive	5003 (TP)	2492 (FN)	7495
	Negative	4535 (FP)	2960 (TN)	7495
Total Predicted		9538	5452	14990

Table 1: Confusion Matrix of the Model Performance

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} = \frac{5003}{5003 + 2492} = 66.75\%$$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP} = \frac{2960}{2960 + 4535} = 39.49\%$$

$$\text{Overall Accuracy} = \frac{TP + TN}{\text{Total}} = \frac{5003 + 2960}{14990} = 53.12\%$$

The model is just above the 50% threshold, i.e., randomness, with a positive bias, in fact, it tends to say it's a good cut in most cases.

The recognition between bad_jump and bad_mix is balanced, but we must consider that the model has seen far fewer bad mixes(only 3000 on about 180 000 global bad cuts), so we can say it has learned to generalize much better about bad mixes.

4.1.2 Last training

For the last test we took new videos that the model has never seen, and although the results are not encouraging there has been an improvement.

		Predicted Class		Total Real
		Positive	Negative	
Actual Class	Positive	44 (TP)	117 (FN)	161
	Negative	22 (FP)	139 (TN)	161
Total Predicted		66	256	322

Table 2: Confusion Matrix of the Model Performance

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} = \frac{44}{44 + 117} = 27.33\%$$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP} = \frac{139}{139 + 22} = 86.34\%$$

$$\text{Overall Accuracy} = \frac{TP + TN}{\text{Total}} = \frac{44 + 139}{322} = 56.83\%$$

The results were given on a smaller number of samples and show a slight improvement, in fact we go from an accuracy of 53% to almost 57%, adding the new videos helped the model slightly to

generalize. With the weight balancing we had the opposite effect, now the model is biased towards negative videos.

Although not the primary objective of the research, the comparison between the model at epoch 2 and that at epoch 14 (of training in Fig 11) revealed a significant trade-off between specialization and generalization. The less trained model (epoch 2) demonstrated a greater ability to correctly identify no-cuts, while the final model (epoch 14) incorrectly classified such segments as bad cuts. This suggests that, as the epochs increased, the model developed a bias toward detecting motion or discontinuities, interpreting any variation as a cut, albeit a poor one. In summary, although the advanced model offers superior accuracy in distinguishing between good and bad cuts, the early model maintains a broader and more conservative view of the scene, resulting in less false positives in the absence of cuts.

5 Analysis and Future Recommendations

This project could have interesting applications in the film industry, particularly in helping video editing avoid human errors by analyzing scenes and flagging when there's any doubt that a cut has been edited incorrectly.

However, the project is still far from practical application, but there are some future avenues for achieving better results.

An interesting direction would be to consider the SlowFast model. SlowFast networks can be described as a single stream architecture that operates at two different framerates Slow pathway and Fast pathway. The Slow pathway can be any convolutional mode that works on a clip of video as a spatiotemporal volume (as R3+1D [30]), in parallel the Fast pathway is another convolutional model with the following properties : high frame rate, high temporal resolution features and low channel capacity.[5] It's great at capturing the pace of the video and the results shown in the search are great.

Furthermore, when parsing the initial training data, such a research could greatly benefit, in our opinion, from a solid, already established database consisting of cuts cleanly chosen and expertly labeled as "Good". The database currently used had clear biases, so building a large database from scratch would be ideal. While such a work might be tedious and could potentially amount to several months or even years, it would be a great benefit to the whole effort of incorporating machine learning technologies into the art of making film. Who's to say that if a model was correctly given a large set of accurately measured cuts, that it would not eventually make decisions that might trump the prominent makers of the time?

6 Conclusion

The ResNet-18 model is very powerful and easily overfits, so it requires a significant amount of data for training (on the order of 100,000 or more). Furthermore, our tests were performed on a batch size of 4 due to performance limitations. The classification of film cuts has proven to be a problem of extreme semantic complexity. Despite the effectiveness of architectures such as ResNet-(2+1)D in capturing spatio-temporal dynamics, a significant gap (Semantic Gap) persists between pixel analysis and the artistic understanding of montage. The difficulty encountered in distinguishing purely aesthetic classes demonstrates that we are faced with a high-level cognitive problem, where the machine must clash with the subjective and creative nature of human work. This work therefore represents a step towards the computational formalization of a language, the filmic one, which still remains deeply anchored to human intuition.

List of Figures

1	Different cut types, source: [21]	3
2	Pixel changes between frames of a scene and the threshold for cut detect	4
3	Creating bad jumps through pruning 5 frames before the original cut, thereby shifting the cut	5
4	Creating bad mix through combining a different shot after the original cut	5

5	(2+1)D vs 3D convolution. The illustration is given for the simplified setting where the input consists of a spatiotemporal volume with a single feature channel. (a) Full 3D convolution is carried out using a filter of size $t \times d \times d$ where t denotes the temporal extent and d is the spatial width and height. (b) A (2+1)D convolutional block splits the computation into a spatial 2D convolution followed by a temporal 1D convolution. We choose the numbers of 2D filters (M_i) so that the number of parameters in our (2+1)D block matches that of the full 3D convolutional block.[30]	6
6	Training with building configuration on First Dataset.	7
7	Training on the Second Dataset with the building configuration	7
8	Training on the Second Dataset with Data Augmentation	8
9	Training on new dataset with new No_cuts class	9
10	Training on the Second Dataset with Dropout	9
11	Training on all Dataset	10

References

- [1] Eashan Adhikarla, Kai Zhang, Jun Yu, Lichao Sun, John Nicholson, and Brian D. Davison. Robust computer vision in an ever-changing world: A survey of techniques for tackling distribution shifts, 2023.
- [2] Perpetua Ogechi Aondover. Evolution of cinematography in the streaming age: challenges and opportunities for filmmakers. *Feedback International Journal of Communication*, 2(1):14–26, 2025.
- [3] Shaofeng Cai, Yao Shu, Gang Chen, Beng Chin Ooi, Wei Wang, and Meihui Zhang. Effective and efficient dropout for deep convolutional neural networks, 2020.
- [4] Ivan Culjak, David Abram, Tomislav Pribanic, Hrvoje Dzapov, and Mario Cifrek. A brief introduction to opencv. In *2012 proceedings of the 35th international convention MIPRO*, pages 1725–1730. IEEE, 2012.
- [5] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition. *CoRR*, abs/1812.03982, 2018.
- [6] Yang Feng, Futang Peng, Xu Zhang, Wei Zhu, Shanfeng Zhang, Howard Zhou, Zhen Li, Tom Duerig, Shih-Fu Chang, and Jiebo Luo. Unifying specialist image embedding into universal image embedding. *arXiv preprint arXiv:2003.03701*, 2020.
- [7] Noa Garcia and Yuta Nakashima. Knowledge-based video question answering with unsupervised scene descriptions. In *European conference on computer vision*, pages 581–598. Springer, 2020.
- [8] Abhijit Ghosh. Analysis of single-shot and long-take filmmaking: its evolution, technique, mise-en-scene, and impact on the viewer. *Indian Journal of Mass Communication and Journalism*, 2(2):4–12, 2022.
- [9] Rohit Girmaji, Sudheer Achary, Adhiraj Deshmukh, and Vineet Gandhi. Assessing active speaker detection algorithms through the lens of automated editing. In *Proceedings of the 2023 ACM International Conference on Interactive Media Experiences Workshops*, pages 123–130, 2023.
- [10] Ahmed Hassanien, Mohamed Elgharib, Ahmed Selim, Sung-Ho Bae, Mohamed Hefeeda, and Wojciech Matusik. Large-scale, fast and accurate shot boundary detection through spatio-temporal convolutional neural networks. *arXiv preprint arXiv:1705.03281*, 2017.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
- [12] Will Kay, João Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, Mustafa Suleyman, and Andrew Zisserman. The kinetics human action video dataset. *CoRR*, abs/1705.06950, 2017.
- [13] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014.

- [14] Roshan Kumari and Saurabh Kr Srivastava. Machine learning: A review on binary classification. *International Journal of Computer Applications*, 160(7), 2017.
- [15] Mackenzie Leake, Abe Davis, Anh Truong, and Maneesh Agrawala. Computational video editing for dialogue-driven scenes. *ACM Trans. Graph.*, 36(4):130–1, 2017.
- [16] Shaopeng Liu, Guohui Tian, and Yuan Xu. A novel scene classification model combining resnet based transfer learning and data augmentation with a filter. *Neurocomputing*, 338:191–206, 2019.
- [17] Jakub Lokoč, Gregor Kovalčík, Tomáš Souček, Jaroslav Moravec, and Přemysl Čech. A framework for effective known-item search in video. In *Proceedings of the 27th ACM International Conference on Multimedia*, pages 1777–1785, 2019.
- [18] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101, 2017.
- [19] Gustavo Mercado. *The filmmaker’s eye: Learning (and breaking) the rules of cinematic composition*. Routledge, 2013.
- [20] Jan Newmarch. Ffmpeg/libav. In *Linux sound programming*, pages 227–234. Springer, 2017.
- [21] Alejandro Pardo, Fabian Caba Heilbron, Juan León Alcázar, Ali K. Thabet, and Bernard Ghanem. Moviecuts: A new dataset and benchmark for cut type recognition. *CoRR*, abs/2109.05569, 2021.
- [22] Zeeshan Rasheed and Mubarak Shah. Detection and representation of scenes in videos. *IEEE transactions on Multimedia*, 7(6):1097–1105, 2005.
- [23] Yuma Sasaka, Takahiro Ogawa, and Miki Haseyama. A novel framework for estimating viewer interest by unsupervised multimodal anomaly detection. *IEEE Access*, 6:8340–8350, 2018.
- [24] Jiaxin Shi, Hanwang Zhang, and Juanzi Li. Explainable and explicit visual reasoning over scene graphs. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8376–8384, 2019.
- [25] Tim J Smith and John M Henderson. Edit blindness: The relationship between attention and global change blindness in dynamic scenes. *Journal of eye movement research*, 2(2), 2008.
- [26] Tomáš Soucek and Jakub Lokoc. Transnet v2: An effective deep network architecture for fast shot transition detection. corr abs/2008.04838 (2020). *arXiv preprint arXiv:2008.04838*, 2020.
- [27] Shitao Tang, Litong Feng, Zhanghui Kuang, Yimin Chen, and Wei Zhang. Fast video shot transition localization with deep structured models. In *Asian Conference on Computer Vision*, pages 577–592. Springer, 2018.
- [28] René Marcelino Abritta Teixeira, Toshihiko Yamasaki, and Kiyoharu Aizawa. Determination of emotional content of video clips by low-level audiovisual features: A dimensional and categorial experimental approach. *Multimedia Tools and Applications*, 61(1):21–49, 2012.
- [29] Suramya Tomar. Converting video formats with ffmpeg. *Linux journal*, 2006(146):10, 2006.
- [30] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition. *CoRR*, abs/1711.11248, 2017.
- [31] Hui-Yin Wu and Marc Christie. Analysing cinematography with embedded constrained patterns. In *WICED-Eurographics Workshop on Intelligent Cinematography and Editing*, 2016.
- [32] Hui-Yin Wu, Francesca Palù, Roberto Ranon, and Marc Christie. Thinking like a director: Film editing patterns for virtual cinematographic storytelling. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 14(4):1–22, 2018.
- [33] Zhen Yang, Ming Ding, Tinglin Huang, Yukuo Cen, Junshuai Song, Bin Xu, Yuxiao Dong, and Jie Tang. Does negative sampling matter? a review with insights into its theory and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(8):5692–5711, 2024.
- [34] Xinrong Zhang, Yanghao Li, Yuxing Han, and Jiangtao Wen. Ai video editing: a survey. 12 2021.